Tema 2

Manejo de la sintaxis del lenguaje



ÍNDICE

01 Variables

02 Tipos de datos

03 Asignaciones

04 Operadores

05 Comentarios al código

06 Sentencias

07 Decisiones

08 Bucles

09 Pruebas

Variables

- Una variable es un nombre que apunta a un valor.
- Pueden declararse con **const** o con **let, var** no está recomendada.
  - **const**: para valores inmutables.
  - **let**: para valores mutables.
- El **ámbito** es la zona del código donde una variable existe. En el caso de **let** y **const**, ambas tienen ámbito de bloque, por lo que solo existen dentro de las llaves donde se declaran.

Variables: var, let y const

Declaración	Ámbito	¿Reasignable?	¿Redeclarable?	Antes de su declaración
var	Función	Sí	Sí	Existe y vale undefined (hoisting)
let	Bloque	Sí	No	Error (zona muerta temporal)
const	Bloque	No	No	Error (zona muerta temporal)

➔ **const** impide reasignar la variable, no modificar su contenido: **const lista = [];**

lista.push(1); es válido.

Variables: locales y globales

- **Local**: declarada dentro de una función (o de un bloque, con let/const). Solo existe ahí y se destruye al salir.
- **Global**: declarada fuera de cualquier función. Es accesible desde todo el script.
- **Global implícita**: asignar un valor sin declarar la variable crea una global. En modo estricto ("use strict") es un error.

```
let contador = 0;              // global

function sumar() {
  let paso = 1;                // local
  total = contador + paso;     // global implícita
  contador = total;
}

sumar();
console.log(contador);         // 1
console.log(paso);             // ReferenceError
```

Literales

➔ Un literal es un valor escrito directamente en el código. A diferencia de una variable, su valor es fijo.

Tipo de literal	**Ejemplos**
Número	42 3.14 6.023e23 0xFF
Cadena	"hola" 'hola' `hola ${nombre}`
Booleano	true false
Array	[1, 2, 3]
Objeto	{ nombre: "Ana", edad: 27 }
Nulo	null
Expresión regular	/^\d{5}$/

Variables: nombres

- Pueden contener letras, dígitos, guion bajo (_) y dólar ($), pero no pueden empezar por un dígito.
- No pueden contener espacios ni coincidir con una palabra reservada del lenguaje.
- JavaScript distingue mayúsculas de minúsculas: **edad**, **Edad** y **EDAD** son tres variables distintas.
- Por convención se usa camelCase: **fechaNacimiento**, **totalConIva**.
- Se pueden declarar varias en una línea separadas por comas. Las que no reciben valor valen **undefined**.

```
let nombre = "Ana", edad, activo = true;
console.log(edad);   // undefined
let precio$ = 10;    // válido
let 2dias = 5;       // SyntaxError: no puede empezar por un dígito
```

Variables: palabras reservadas

`break`	`case`	`catch`	`class`	`const`	`continue`	`debugger`
`default`	`delete`	`do`	`else`	`export`	`extends`	`false`
`finally`	`for`	`function`	`if`	`import`	`in`	`instanceof`
`new`	`null`	`return`	`super`	`switch`	`this`	`throw`
`true`	`try`	`typeof`	`var`	`void`	`while`	`with`
`yield`	`let`	`static`	`await`	`enum`		

➔ **let**, **static**, **yield** y **await** solo están reservadas en ciertos contextos (modo estricto, módulos, funciones asíncronas o generadoras), pero conviene evitarlas siempre.

Tipos de datos

- JavaScript tiene **tipado dinámico**: no declaras el tipo de una variable, sino que cada valor tiene el suyo y una misma variable puede guardar primero un número y después un texto.
- Hay 7 tipos **primitivos** (valores simples e inmutables) y el tipo **objeto**, que agrupa todo lo demás (objetos, arrays, funciones, fechas, etc.).

Tipos de datos

Tipo	Ejemplos	typeof	A tener en cuenta
number	42, 3.14, -7, NaN, Infinity	“number”	Enteros y decimales son el mismo tipo (coma flotante de 64 bits
bigint	9007199254740993n	“bigint”	Enteros de cualquier tamaño; no se mezclan con number sin convertir
string	"hola", 'hola', `hola ${nombre}`	“string”	Inmutables: los métodos devuelven textos nuevos
boolean	true, false	“boolean”	El resultado de cualquier comparación
undefined	undefined	“undefined”	Variable sin valor, propiedad inexistente, función sin return
null	null	“object”	«Vacío a propósito». Lo de «object» es un error histórico
symbol	Symbol("id")	“symbol”	Identificadores únicos, sobre todo en librerías
object	{ nombre: "Ana" }, [1, 2], new Date()	“object”	Arrays incluidos: para distinguirlos usa Array.isArray(x)
(function)	function f() {}	“function”	Las funciones son objetos que se pueden llamar

Cadenas: comillas y escapes

- Se delimitan con comillas dobles, simples o con backticks (plantillas).
- Dentro de unas comillas se pueden usar las otras sin escapar.
- Para lo demás se usan los caracteres de escape, que empiezan por ****.

Escape	Significado
\n	Salto de línea
\t	Tabulador
\'	Comilla simple
\"	Comilla doble
\\	Contrabarra
\r	Retorno de carro
\b	Retroceso
\f	Avance de página
\u00F1	Carácter Unicode (ñ)

Números: notación y bases

Notación	Ejemplo	Valor
Decimal	255	255
Exponencial	6.023e23 1.5e-3	$6{,}023 \times 10^{23}$ 0,0015
Hexadecimal	0xFF	255
Octal	0o377	255
Binario	0b11111111	255

- El octal antiguo con cero inicial (**0377**) está obsoleto y da error en modo estricto.
- Todos son el mismo tipo **number**: coma flotante de 64 bits.
- Por eso **0.1 + 0.2** da **0.30000000000000004** y los enteros solo son exactos hasta **Number.MAX_SAFE_INTEGER** ($2^{53} - 1$).

Arrays y objetos

➔ **Array:**

- Lista ordenada entre **[ ]**, separada por comas.
- Índices desde 0; **length** da el tamaño.
- Puede mezclar valores de distintos tipos.

```
const dias = ["lunes", "martes", "miércoles"];
dias[0];        // "lunes"
dias.length;    // 3
dias[5];        // undefined
```

➔ **Objeto:**

- Pares **clave**: **valor** entre **{ }**.
- Acceso con punto o con corchetes.

```
const avion = {
  marca: "Boeing",
  modelo: "747",
  pasajeros: 450
};
avion.modelo;      // "747"
avion["marca"];    // "Boeing"
```

Conversión implícita

➔ JavaScript convierte los tipos automáticamente según el operador.

➔ Con + y un string, concatena. Con -, *, / y %, convierte a número.

Expresión	Resultado	Motivo
"5" + 2	"52"	+ con un string concatena
"5" - 2	3	- convierte a número
"5" * "2"	10	* convierte ambos a número
true + 1	2	true se convierte en 1
"abc" * 2	NaN	"abc" no es un número válido
1 + 2 + "3"	"33"	Se evalúa de izquierda a derecha
"5" == 5	true	== convierte antes de comparar
"5" === 5	false	=== no convierte

Conversión explícita

Función	Ejemplo	Resultado
parseInt(texto, base)	parseInt("42px")	42
	parseInt("ff", 16)	255
parseFloat(texto)	parseFloat("3.14abc")	3.14
	parseFloat("abc")	NaN
Number(valor)	Number("42px")	NaN
num.toString(base)	(13).toString(16)	"d"
String(valor)	String(true)	"true"

- **parseInt** y **parseFloat** leen hasta el primer carácter no válido. **Number** exige que todo el texto sea numérico.
- La base de **parseInt** va de 2 a 36. Conviene indicarla siempre.
- Para comprobar si falló: **Number.isNaN(x)**.

Asignaciones

➔ El operador = se utiliza para asignar valores a las variables.

```
let x = 4
let y = 5
```

Operador	Descripción
=	Asigna a la variable de la parte izquierda el valor de la parte derecha.

Operadores aritméticos

➔ Se utilizan para llevar a cabo cálculos aritméticos.

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo

Operadores de comparación

➔ Se usan para comparar dos valores entre sí.

➔ El valor de retorno siempre es un booleano: *true* o *false*.

Operador	Descripción
===	Compara dos elementos, incluyendo su tipo interno. Si son de distinto tipo, no realiza conversión y devuelve false ya que siempre los considera diferentes. Uso recomendado.
!==	Compara dos elementos, incluyendo su tipo interno. Si son de distinto tipo, no realiza conversión y devuelve true ya que siempre los considera diferentes. Uso recomendado.
==	Devuelve el valor true cuando los dos operandos son iguales. Si los elementos son de distintos tipos, realiza una conversión. No está recomendado su uso.
!=	Devuelve el valor true cuando los dos operandos son distintos. Si los elementos son de distintos tipos, realiza una conversión. No está recomendado su uso.

Operadores de comparación

- Se usan para comparar dos valores entre sí.
- El valor de retorno siempre es un booleano: *true* o *false*.

Operador	Descripción
>	Devuelve el valor true cuando el operando de la izquierda es mayor que el de la derecha.
<	Devuelve el valor true cuando el operando de la izquerda es menor que el de la derecha.
>=	Devuelve el valor true cuando el operando de la izquierda es mayor o igual que el de la derecha.
<=	Devuelve el valor true cuando el operando de la izquierda es menor o igual que el de la derecha.

Operadores de incremento

➔ Se utilizan para aumentar o disminuir en 1 el valor de una variable.

Operador	Descripción
++	Incrementa en 1 la variable.
- -	Disminuye en 1 la variable.

➔ **IMPORTANTE:** En caso de asignación con incremento, es importante el orden de la variable incrementada. No es lo mismo **y = ++x** que **y = x++**.

Operadores aritméticos abreviados

➔ Se utilizan para llevar a cabo cálculos aritméticos de manera abreviada.

Operador	Descripción
+=	Suma los operandos izquierdo y derecho y asigna el resultado al operando izquierdo.
-=	Resta el operando derecho del operando izquierdo y asigna el resultado al operando izquierdo.
*=	Multiplica ambos operandos y asigna el resultado al operando izquierdo.
/=	Divide ambos operandos y asigna el resultado al operando izquierdo.

Operadores booleanos

➔ Se utilizan para procesar valores booleanos.

➔ Devuelven valores booleanos: *true* o *false*.

Operador	**Descripción**
&&	AND: Devuelve *true* cuando ambos operandos son verdaderos.
||	OR: Devuelve *true* cuando alguno de los operandos es verdadero o ambos lo son.
!	NOT: Devuelve *false* si el valor es *true* y *true* cuando el valor es *false*.

Operadores bit a bit

➔ Trabajan sobre la representación binaria de los números como enteros de 32 bits.

Operador	Nombre	Ejemplo	Resultado
&	AND	5 & 3	0101 & 0011 = 0001
|	OR	5 | 3	0101 | 0011 = 0111
^	XOR	5 ^ 3	0101 ^ 0011 = 0110
~	NOT	~5	−(5 + 1)
<<	Desplaz. izquierda	43 << 3	Multiplica por 2^3
>>	Desplaz. derecha	-43 >> 3	Conserva el signo
>>>	Desplaz. sin signo	-1 >>> 28	Rellena con ceros

Otros operadores

Operador	Uso	Ejemplo
+	Concatenar cadenas	"Hola, " + nombre
? :	Condicional (ternario)	x > 0 ? "positivo" : "no positivo"
. y [ ]	Acceso a un miembro	avion.modelo avion["modelo"]
,	Varias expresiones; devuelve la última	for (let i = 0, j = 9; i < j; i++, j--)
new	Crear un objeto	new Date()
delete	Eliminar una propiedad	delete avion.modelo
typeof	Obtener el tipo	typeof 42 → "number"
instanceof	Comprobar la clase	fecha instanceof Date
in	Comprobar si existe una propiedad	"modelo" in avion
void	Evaluar y devolver undefined	void 0

Precedencia de operadores

➔ Trabajan sobre la representación binaria de los números como enteros de 32 bits.

Nivel	Operadores
1	( ) agrupación
2	. [ ] new( ) llamada( )
3	x++ x--
4	! ~ +x -x ++x --x typeof void delete
5	**
6	* / %
7	+ -
8	<< >> >>>
9	< <= > >= in instanceof

Nivel	Operadores
10	== != === !==
11	&
12	^
13	|
14	&&
15	|| ??
16	? :
17	= += -= *= ...
18	,

Expresiones

- ➔ Una expresión combina variables, literales y operadores, y produce un valor.
- ➔ Puede asignar un valor (x = 5) o calcularlo (4 + y).

Tipo	Ejemplo	Resultado
Aritmética	(2 + 4 - 2) * 10	40 (number)
Lógica / relacional	x >= 5 && x <= 8	true o false
De cadena	"Hola, " + nombre	Un string
De asignación	x = 5	5 (el valor asignado)

Tablas de verdad

p	q	p&&q
V	V	V
V	F	F
F	V	F
F	F	F

p	q	p||q
V	V	V
V	F	V
F	V	V
F	F	F

p	q
V	F
F	V

➔ En JavaScript, **&&** y **||** se evalúan en cortocircuito y devuelven uno de los operandos, no necesariamente un booleano: **"" || "anónimo"** da **"anónimo"**.

Comentarios al código

- Sirven para documentar brevemente el código.
- Pueden ser de dos tipos:
  - De una línea -> //
  - De varias líneas -> /* */

Sentencias

- Son instrucciones que realizan determinadas acciones.
- Pueden declarar una variable, ser llamadas a funciones, etc.
- Una sentencia termina con un punto y coma. Aunque se inserta por defecto con ASI (*Automatic Semicolon Insertion*), añadirlo explícitamente puede evitar posibles errores.
- Una función agrupa instrucciones que se repiten para reutilizarlas llamándola por su nombre.
- Puede recibir parámetros y devolver un valor con return.
- Se verán en detalle en el Tema 4.

Decisiones: if/else

- La instrucción if/else permite la ejecución de un bloque u otro de instrucciones en función de una condición.
- En el ejemplo, en lugar de usar **else if** también podrían haberse anidado las condiciones.
- **Sintaxis:**

```
if (condition) {
    // Instrucciones si "condicion" se cumple
} else if (otherCondition) {
    // Instrucciones si "condition" no se ha cumplido, pero "otherCondition" sí
} else {
    // Instrucciones si ninguna condición se ha complido
}
```

Decisiones: switch

- Se define cada caso con un **break**.
- Sin **break** se continúa en el siguiente caso, por lo que comparten instrucciones.
- **Sintaxis:**

```
switch(variable) {
    case "x":
        // Instrucciones del caso particular
    break;
}
```

Switch: default y casos agrupados

- **default** se ejecuta si ningún case coincide. Es opcional y suele ir al final.
- Los case se comparan en orden y con igualdad estricta (===).
- Varios case seguidos sin break comparten instrucciones.

```
switch (dia) {
  case "sábado":
  case "domingo":
    tipo = "fin de semana";
    break;
  case "lunes":
    tipo = "inicio de semana";
    break;
  default:
    tipo = "laborable";
}
```

Bucle for

➔ Cuando el programa llega a un bucle **for**:

- Se inicializa el índice. Esto solo se hace una vez.
- Se analiza la condición de prueba y, en caso de cumplirse, se ejecutan las instrucciones del bucle.
- Cuando termina la ejecución de las instrucciones del bucle, se modifica el índice, se vuelve a la cabecera del bucle en ejecución y se lleva a cabo la condición de prueba de nuevo.
- Si la condición se cumple, se ejecutan las instrucciones y, si no, continua con las líneas de código posteriores al bucle.

Bucle for

➔ **Sintaxis:**

```
for (indexInicialization; condition; indexModifier) {
    // Instrucciones del bucle
}
```

for...in y for...of

	for...in	for...of
Recorre	Las claves (nombres de propiedad)	Los valores
Se usa con	Objetos	Arrays, strings, Map, Set...
En un array da	Los índices como texto: "0", "1"...	Los elementos
Recorre	Las claves (nombres de propiedad)	Los valores

```
const avion = { marca: "Boeing", modelo: "747" };
for (const clave in avion) {
  console.log(clave, avion[clave]);
}
```

```
const dias = ["lunes", "martes"];
for (const dia of dias) {
  console.log(dia);
}
```

➔ Para arrays, mejor **for...of** (o **forEach**): **for...in** también recorre propiedades heredadas.

Bucle while

- Sirve para la ejecución de instrucciones mientras se cumpla una condición determinada:
  - Si la condición nunca se cumple, no se ejecuta ninguna instrucción.
  - Si la condición se cumple siempre, será un bucle infinito.
    - Es importante que la condición deje de cumplirse en cierto punto para evitarlos.
- **Sintaxis:**

```
while (condition) {
    // Instrucciones del bucle
}
```

Bucle do-while

➔ A diferencia del bucle **while**, el bucle **do-while** no comprueba la condición hasta el final, por lo que el cuerpo de instrucciones se recorrerá al menos una vez.

- Si la condición no se cumple, sale del bucle.
- Si se cumple, vuelven a ejecutarse las instrucciones.

➔ **Sintaxis:**

```
do {
    // Instrucciones del bucle
} while(condition);
```

Bucles: instrucciones *break* y *continue*

➔ Dentro de un bucle, podemos usar las instrucciones **break** y **continue** de la siguiente manera:

- **break:** Interrumpe la ejecución del bucle aunque no se haya ejecutado por completo. El programa sigue a continuación del bucle.
- **continue:** Retorna a la cabecera del bucle, por lo que se ejecuta de nuevo la condición (y se incrementa el índice si es un bucle **for**).

Pruebas

➔ **Pruebas:**

- **DevTools:** usando la pestaña **Consola** en el navegador (Google Chrome, Firefox Developer Edition, etc.).
- **Breakpoints:** usando puntos de ruptura para depurar el código.

➔ **Documentación:**

- Comentarios en el código con los formatos vistos anteriormente y el formato JSDoc.